

DSI Development Programme: Consolidated Architecture & Phase Plan

Item	Value
Version	1.0
Date	April 2026
Classification	Master Development Plan
Purpose	Single reference document for Claude Code execution of all pending DSI development

HOW TO USE THIS DOCUMENT

This document contains everything needed to write detailed phase development plans for DSI. It is structured in four parts:

1. **Context** — What DSI is, how the architecture works, what exists in the codebase
2. **Workstream WE** — The World Engine (5 phases, ready to execute)
3. **Workstreams A, B, C** — Authentication, Admin Backend, and Experience-Based Recalibration (10 phases, requiring detailed phase documents)
4. **Execution Plan** — Dependency chain, sequencing, and priorities

The task for Claude Code: Read this document, then write detailed phase development plans for the 10 phases in Workstreams A, B, and C. Each phase document must match the format and level of detail shown in the WE phases (Part 2) and in the reference phase document (Appendix A). The WE phases are already complete and do not need to be rewritten.

Before writing any phase document: Read the project's SKILL.md and the relevant existing code to understand current implementation state. Phase documents must reference specific files, provide code signatures, and include success criteria.

PART 1: CONTEXT

1.1 What is DSI?

Digital Signal Intelligence (DSI) is a new insurance information substrate based on observable digital signals rather than self-reported documentation. It produces composite risk scores, loss propensity, exposure estimates, and pricing decisions from 200-400 signals per entity across seven canonical categories.

Tech stack: Python 3.10+, FastAPI, SQLAlchemy, PostgreSQL, Redis, Next.js/React frontend, Rust (PyO3) for performance-critical graph operations.

Codebase structure:

```
digital-signal-intelligence/
├─ signal_architecture/      # Signal processing (extractors, aggregators, ca
├─ infrastructure/          # API (FastAPI), DB (SQLAlchemy), analytics, bui
├─ layers/                  # Assessment layers (risk/14-step workflow, expo
├─ coverages/               # YAML configs (10 coverages, 76 sub-configurati
├─ commercial/entities/     # Commercial entity YAML definitions
├─ world_engine/            # (NEW – to be built per WE phases)
├─ frontend/                # Next.js React workbench UI
├─ rust/dsi-core/           # PyO3 crate (PageRank, derivatives, validation)
├─ tests/                   # 380+ tests
├─ deploy/                  # Docker, Kubernetes, monitoring
├─ development/             # Phase development plans
└─ SKILL.md                 # Architecture guide (READ THIS FIRST)
```

1.2 The Three-Layer Architecture

DSI separates concerns into three abstraction layers:

Layer 1: Coverage Configuration (`coverages/{coverage}/config.yaml`)

- **Responsibility:** Technical pricing — signals, weights, tiers, score_conditions, ILF curves, deductible factors
- **Owned by:** Product / Actuarial team
- **Change cadence:** Regular (tuning, new signals)
- **Formula:** $P = (B \times R) \times ILF \times D \times M_{risk} \times M_{loss} \times M_{exp}$

Layer 2: Commercial Entity (`commercial/entities/{entity}.yaml`)

- **Responsibility:** Distribution economics — appetite, commission, taxes, FX, fronting, discretion
- **Owned by:** Distribution / Underwriting Management
- **Change cadence:** Occasional (new entities, authority changes)
- **Key principle:** The pricing engine produces a technical premium in USD. The commercial entity transforms it into a gross/offered premium in local currency.

Layer 3: World Engine (`world_engine/` — to be built)

- **Responsibility:** Autonomous causal discovery, cross-signal validation, emergent intelligence, portfolio simulation
- **Owned by:** Nobody — fully autonomous, self-calibrating
- **Change cadence:** Continuous — no human configuration
- **Key principle:** Publishes intelligence to a registry. Lower layers can consume but are not required to obey.

1.3 Existing Authentication

The API has JWT + API key authentication (`infrastructure/api/auth/`). This is lightweight and not enterprise-grade. No multi-tenant support. No SSO/SAML. No comprehensive audit trail. No real-time data synchronisation across connected clients.

1.4 Existing Admin Capabilities

There is no admin backend. The following operational tools exist but are CLI-only:

- `python -m layers.risk.calibration_harness <coverage>` — Validates pricing configs
- `python coverages/doc_generator.py` — Regenerates config documentation
- `python development/project/assessments/scripts/assess_project.py` — Project assessment
- `python -m infrastructure.builder.cli validate` — Config validation
- Prometheus metrics at `/api/v1/metrics` (request latency, error rates, workflow duration)

- Health checks at `/api/v1/health/live` and `/api/v1/health/ready`

No frontend admin interface exists for any of these capabilities.

1.5 Existing Recalibration

The Configuration Architecture document defines a separation between:

- **Config** (design decisions): Signal definitions, weights, correlation direction, band thresholds
- **Analysis** (empirical findings): `lag_months`, normaliser coefficients, confidence intervals, sample sizes

Analysis outputs live in `analysis_outputs/{coverage}_v{date}.yaml`. However:

- No automated process produces these analysis outputs from actual loss data
- No pipeline connects loss events to signal profiles at bind time
- No proposal/approval workflow for recalibration exists
- No UI surfaces the recalibration process
- The calibration harness validates pricing sanity but does not perform experience-based recalibration

1.6 Key Existing Files Referenced Across Phases

Component	Location	Relevance
SKILL.md	Root	Architecture guide — READ FIRST before any work
Foundational Principles	<code>docs/overview/Foundational Principles.md</code>	Core DSI principles (must follow)
Config Schema	<code>coverages/master_config_layout.yaml v2.3</code>	YAML config structure reference
Commercial Entity	<code>docs/overview/Commercial_Entity_Schema.md</code>	Entity YAML

Schema		structure
Configuration Architecture	<code>docs/overview/Configuration_Architecture.md</code>	Layer separation, analysis outputs
Premium Calculation	<code>docs/overview/Premium_Calculation_Methodology.md</code>	Pricing formula, assembly pipeline
Referral Methodology	<code>docs/overview/Referral_Methodology.md</code>	Signal override vs contextual decision
API Auth	<code>infrastructure/api/auth/</code>	Current JWT + API key auth
API Main	<code>infrastructure/api/main.py</code>	FastAPI app, router mounting
DB Models	<code>infrastructure/db/models.py</code>	SQLAlchemy models
Workflow	<code>layers/risk/workflow.py</code>	14-step pricing workflow
Scorer	<code>layers/risk/scorer.py</code>	Steps 4-6, <code>score_conditions</code>
Pricer	<code>layers/risk/pricer.py</code>	Steps 8-12, premium calculation
Config Validator	<code>infrastructure/validation/config_validator.py</code>	YAML config validation
Calibration Harness	<code>layers/risk/calibration_harness.py</code>	Pricing sanity checking
Premium Assembly	<code>layers/risk/premium_assembly.py</code>	Technical → commercial premium
Analytics	<code>infrastructure/analytics/</code>	Performance, portfolio, signal

		analytics
Graph Runtime	<code>signal_architecture/graph/</code>	Organisational Graph (6 node, 6 edge, 5 derivatives)
Frontend Store	<code>frontend/src/stores/dsiStore.ts</code>	Zustand state management
Frontend Layout	<code>frontend/src/app/layout.tsx</code>	App layout, sidebar navigation
Seed Script	<code>seed_dsi_bench.py</code>	Seeds database with realistic end-to-end data

1.7 Phase Document Format

Every phase document must follow this structure (see Appendix A for a full example):

1. **Overview** — One paragraph explaining what is being built and why
2. **Current State** — What exists in the codebase today (specific files, modules, tables)
3. **Target State** — What will exist after the phase is complete
4. **File Structure** — New directories and files being created
5. **Implementation Plan** — Sub-phases (a, b, c...) each specifying:
 - Files to create or modify (exact paths)
 - Code signatures (class/function definitions with docstrings)
 - Database changes (migrations, tables, indexes)
 - API endpoints (method, path, description)
 - Frontend components (new files, data sources, content sections)
6. **Constraints & Principles** — Rules that must not be violated
7. **Dependencies** — What must be complete before this phase starts
8. **Success Criteria** — Specific, testable conditions that prove the phase is complete

PART 2: WORLD ENGINE PHASES (COMPLETE — DO NOT REWRITE)

The following 5 phases are fully specified and ready to execute. They are included here for reference and dependency mapping. Do not rewrite these — they are done.

World Engine Architecture Summary

The World Engine is DSI's third architectural layer. It is autonomous, self-calibrating, and structurally independent from both the Coverage Configuration and Commercial Entity layers.

What it does: Consumes the output of every assessment across all entities, all coverages, and all time. Discovers cross-signal causal relationships that were not designed into the system. Validates them statistically without human authorisation. Publishes intelligence to a read-only registry that lower layers can consume.

Key outputs:

- **Discovered Relationships:** Directional causal assertions between signals, with full statistical evidence and autonomous lifecycle (Candidate → Provisional → Active → Deprecated)
- **Consistency Scores:** Per-assessment cross-signal and cross-layer consistency metrics
- **Causal Adjustment Factor (CAF):** Per-assessment forward risk trajectory that enters the premium formula as a parallel pricing dimension: $P_{final} = P_{static} \times CAF$
- **Portfolio Intelligence:** Portfolio Graph assembly, concentration detection, scenario simulation

Pricing Integration (CAF)

The static pricing formula is not modified. The World Engine runs a parallel causal graph assessment that produces a Causal Adjustment Factor. The audit trail shows both: "Static scoring said Tier 2 at £20,250. Causal graph said 1.18x adjustment based on active precursors. Combined premium £23,895."

CAF constraints: floor 0.80, cap 1.50, confidence gate 0.6, minimum 2 relationships evaluated. Defaults to 1.0 when Engine has insufficient data.

Portfolio Simulation

Portfolio Graph assembles individual Organisational Graphs into a composite with inter-entity edges (supply chain, technology, governance, causal pathway correlation). Concentration intelligence feeds into appetite evaluation. Scenario Simulation API propagates hypothetical shocks through the causal graph.

WE Phase Summary

Phase	Name	Scope	Effort
WE-1	Foundation	Types, DB schema, intelligence registry, API	1-2 weeks
WE-2	Consistency Scorer	Inline consistency scoring, frontend card	~2 weeks
WE-3	Discovery & Validation	Correlation scanner, Granger causality, holdout, lifecycle manager	3-4 weeks
WE-4	Causal Graph Pricing	CAF engine, pricing integration, frontend display	3 weeks
WE-5	Portfolio Intelligence	Portfolio Graph, concentration, scenario simulation, dashboard	5-6 weeks

Full phase documents: See the separate “World Engine Phase Development Plan” document for complete specifications of all 5 phases.

PART 3: NEW WORKSTREAMS

Workstream A: Authentication, Audit Trail & Real-Time Sync

Scope Decisions (Confirmed)

- **Auth level:** Full multi-tenant with SSO/SAML — enterprise-ready
- **Audit trail:** Comprehensive — every user action captured with before/after state
- **Real-time sync:** WebSocket-based — changes by one user reflected to all connected clients

Phase A-1: Auth Foundation

What: Replace lightweight JWT auth with enterprise-grade multi-tenant authentication supporting SSO/SAML.

Backend scope:

- User model: id, email, name, tenant_id, role, created_at, last_login, is_active, mfa_enabled
- Tenant model: id, name, slug, sso_provider, sso_metadata, settings, created_at
- Role model: id, name, permissions (JSONB), tenant_id — roles are tenant-scoped
- Permission system: granular permissions (e.g., `assessment:read`, `assessment:write`, `config:read`, `config:write`, `admin:users`, `admin:system`)
- SSO/SAML integration: Support SAML 2.0 via `python3-saml` or similar. SAML metadata exchange, assertion parsing, attribute mapping to DSI user/role model. Also support OIDC for providers like Auth0/Okta/Azure AD.
- Session management: JWT tokens with tenant context, refresh token rotation, configurable session duration per tenant
- API middleware: Replace current auth middleware with tenant-aware version. Every API request carries tenant context. Database queries are tenant-scoped by default.
- Migration: Create users, tenants, roles, permissions tables. Seed default admin user and system tenant.

Frontend scope:

- Login page: Email/password form with SSO redirect button (provider-specific branding)
- MFA setup and verification flow
- Session expiry handling with graceful re-authentication
- Tenant context in all API calls

Key architectural decision: Multi-tenancy is implemented at the database level with a `tenant_id` foreign key on all tenant-scoped tables. This is simpler than schema-per-tenant and sufficient for the expected scale. A middleware layer automatically filters all queries by the authenticated user's tenant.

Dependencies: None — this is the first phase.

Success criteria:

1. Users can log in via username/password and via SAML SSO
 2. MFA enrollment and verification works
 3. API requests without valid auth return 401
 4. Users in Tenant A cannot see data from Tenant B
 5. Role-based permissions control API endpoint access
 6. Default admin user can create new users and tenants
-

Phase A-2: Activity Audit System

What: Comprehensive audit trail capturing every user action, plus WebSocket-based real-time sync so all connected clients see changes immediately.

Backend scope:

- Audit log model: id, tenant_id, user_id, action_type (enum), resource_type, resource_id, before_state (JSONB), after_state (JSONB), metadata (JSONB — IP, user agent, session), timestamp
- Action types: LOGIN, LOGOUT, ASSESSMENT_VIEW, ASSESSMENT_CREATE, REFERRAL_DECISION, SIGNAL_OVERRIDE, PREMIUM_DISCRETION, CONFIG_VIEW, CONFIG_EDIT, CONFIG_DEPLOY, USER_CREATE, USER_EDIT, PERMISSION_CHANGE, ENTITY_EDIT, RECALIBRATION_PROPOSE, RECALIBRATION_APPROVE, RECALIBRATION_REJECT, SYSTEM_SETTING_CHANGE
- Audit middleware: Intercepts all state-changing API requests, captures before/after state automatically
- WebSocket server: FastAPI WebSocket endpoint for real-time event broadcasting. When User A makes a change, a WebSocket message is broadcast to all connected clients in the same tenant. Message format: { event_type, resource_type, resource_id, user_id, timestamp, summary }
- Session tracking: Track active sessions with heartbeat. Record session duration, pages visited, actions taken. This feeds the "time spent" audit requirement.

Frontend scope:

- WebSocket client: Connects on login, reconnects on disconnect, receives real-time events
- Notification system: Toast notifications when another user makes a relevant change

(e.g., "Jane approved referral for Submission #1234")

- Data refresh: When a WebSocket event indicates a resource the current user is viewing has changed, automatically refresh the data. No stale reads.
- Session activity indicator: Show which users are currently active and what they're viewing (optional — privacy-sensitive)

Dependencies: Phase A-1 (user/tenant model)

Success criteria:

1. Every state-changing API call produces an audit log entry with before/after state
 2. WebSocket broadcasts reach all connected clients in the same tenant within 500ms
 3. Frontend auto-refreshes when underlying data changes
 4. Audit log is queryable by user, action type, resource, and date range
 5. Session duration and page views are tracked
-

Phase A-3: Auth Frontend & Login Experience

What: The complete authentication UI — login screen, MFA flow, role-based navigation, session management.

Frontend scope:

- Login page: Clean, branded login screen with email/password fields and SSO button. Error handling for invalid credentials, locked accounts, expired sessions.
- MFA setup: QR code enrollment for TOTP (Google Authenticator, Authy). Backup codes generation and download.
- MFA verification: 6-digit code entry on login.
- Password management: Reset flow (email link), change password (current + new), password strength requirements.
- Role-based navigation: After login, the app shell adapts to the user's role. Underwriters see the submission workbench. Admins see the admin backend. Actuaries see the recalibration interface. Read-only users see a restricted view. Navigation items are permission-gated.
- Session management: Session timeout warning (5 min before expiry), automatic logout, "remember me" option for trusted devices.

- User profile: View own profile, change password, manage MFA, view own audit log.

Dependencies: Phases A-1 and A-2

Success criteria:

1. Login flow works for username/password and SSO
 2. MFA enrollment and verification works end-to-end
 3. Navigation adapts correctly to user role
 4. Session timeout and re-authentication work gracefully
 5. Password reset flow works end-to-end
-

Workstream B: Admin Backend — Full Operational Visibility

Scope Decisions (Confirmed)

- **All capabilities in scope:** Config management, signal monitoring, pipeline monitoring, user management, audit viewer, system health
- **This is a new top-level section in the frontend**, not a tab within the submission workbench

Phase B-1: System Health & Pipeline Monitoring

What: Real-time dashboard showing the operational health of every DSI subsystem.

Backend scope:

- System health aggregator: New module that queries API health, DB connection pool stats, Redis status, and returns a unified health object
- Extractor health metrics: Per-extractor success rate, error rate, average latency, last successful run, TTL compliance, data freshness score. Source: existing Prometheus metrics + new per-extractor tracking table
- Pipeline metrics: Assessment throughput (per hour/day), latency percentiles (P50/P95/P99), failure rate by coverage, queue depth, average signal extraction time, average pricing time
- Metrics API: New endpoints under `/api/v1/admin/health/` that expose system health, extractor health, and pipeline metrics

- Historical metrics: Store hourly snapshots of key metrics for trend analysis (7 days granular, 90 days daily aggregates)

Frontend scope:

- Admin shell: New top-level route (`/admin`) with its own navigation sidebar, permission-gated to admin role
- System health dashboard: Green/amber/red status indicators for API, DB, Redis, each extractor group. Auto-refresh every 30 seconds.
- Extractor health panel: Table of all extractors with status, success rate, last run, data freshness. Expandable rows with error details. Filter by coverage, status, signal type.
- Pipeline metrics panel: Time-series charts for throughput, latency, and failure rates. Coverage breakdown. Date range selector.
- Alert configuration: Set thresholds for alerts (e.g., extractor error rate > 10%, P95 latency > 5s). Alerts display in the dashboard header.

Dependencies: Phase A-1 (admin role required for access)

Success criteria:

1. Dashboard shows real-time health status for all subsystems
2. Extractor health table shows per-extractor metrics with real data
3. Pipeline metrics charts render with historical data
4. Alerts trigger when thresholds are breached
5. Non-admin users cannot access `/admin` routes

Phase B-2: Config Management

What: UI for viewing, editing, validating, and deploying coverage configurations. This replaces CLI-only config management with a governed, auditable workflow.

Backend scope:

- Config API: CRUD endpoints for coverage configurations under `/api/v1/admin/configs/`
 - `GET /configs` — List all coverages and their configs
 - `GET /configs/{coverage}/{config}` — Get config YAML content

- `PUT /configs/{coverage}/{config}` — Update config (creates a draft version)
- `POST /configs/{coverage}/{config}/validate` — Run config validator, return results
- `POST /configs/{coverage}/{config}/calibrate` — Run calibration harness, return results
- `POST /configs/{coverage}/{config}/deploy` — Promote draft to active (requires approval permission)
- `POST /configs/{coverage}/{config}/rollback` — Revert to previous version
- `GET /configs/{coverage}/{config}/history` — Version history with diffs
- Config versioning: Every config change creates a new version (content-addressable storage already exists). Draft → Validated → Calibrated → Deployed lifecycle.
- Config diff engine: Compute YAML diff between any two versions, highlighting weight changes, signal additions/removals, threshold changes.

Frontend scope:

- Config browser: Tree view of coverages → configs. Shows version, last modified, status (active/draft).
- Config editor: YAML editor with syntax highlighting (Monaco or CodeMirror). Live validation on edit.
- Validation panel: Run validator, display results (errors, warnings) inline.
- Calibration panel: Run calibration harness, display results (pass/fail per config, guardrail hit rate, tier distribution).
- Diff viewer: Side-by-side comparison of any two config versions. Highlighted changes.
- Deployment flow: Draft → Validate → Calibrate → Review diff → Deploy (requires admin approval). Full audit trail.
- Version history: Timeline of config changes with author, timestamp, and diff summary.

Dependencies: Phase A-1 (auth), Phase A-2 (audit trail for config changes)

Success criteria:

1. All 76 configs viewable and editable in the UI
2. Validator runs inline and displays results correctly

3. Calibration harness runs from UI and displays results
 4. Config changes create audited versions with diffs
 5. Deployment requires approval from admin-permissioned user
 6. Rollback restores previous version correctly
-

Phase B-3: User & Permission Management

What: Admin UI for managing users, roles, and permissions within a tenant.

Backend scope:

- User CRUD API: `/api/v1/admin/users/` — Create, read, update, deactivate users. Invite flow (email invitation with registration link).
- Role CRUD API: `/api/v1/admin/roles/` — Create, read, update, delete roles. Each role has a permission set.
- Permission registry: Enumerated permissions covering all DSI functions:
 - `assessment:read, assessment:write, assessment:refer`
 - `config:read, config:write, config:deploy`
 - `entity:read, entity:write`
 - `admin:users, admin:system, admin:audit`
 - `recalibration:view, recalibration:approve`
 - `portfolio:view, portfolio:simulate`
- Default roles: UNDERWRITER (assessment read/write/refer), SENIOR_UNDERWRITER (+ config read), ACTUARIAL (+ config write, recalibration), ADMIN (all), READ_ONLY (read-only across all resources)

Frontend scope:

- User management page: List users with role, status, last login. Create/invite new users. Edit user details and role assignment. Deactivate users.
- Role management page: List roles with permission summary. Create/edit roles with permission checkboxes.
- Invitation flow: Send email invitation, user completes registration.

Dependencies: Phase A-1 (user/role schema)

Success criteria:

1. Admin can create, edit, and deactivate users
 2. Admin can create custom roles with specific permissions
 3. Permission enforcement works — users without `config:write` cannot edit configs
 4. User invitation flow works end-to-end
 5. Audit log captures all user/role changes
-

Phase B-4: Audit Log Viewer

What: Searchable, filterable UI for the audit trail produced by Phase A-2.

Backend scope:

- Audit query API: `/api/v1/admin/audit/` with rich filtering:
 - Filter by user, action type, resource type, resource ID, date range
 - Full-text search on metadata
 - Pagination with cursor-based scrolling
 - Export endpoint (CSV/JSON) for compliance

Frontend scope:

- Audit log page: Table with columns: timestamp, user, action, resource, summary. Expandable rows showing full before/after state diff.
- Filter bar: User selector, action type dropdown, resource type dropdown, date range picker.
- Timeline view: Per-entity or per-user activity timeline showing all actions in chronological order.
- Export: Download filtered audit data as CSV or JSON.
- Quick filters: "My actions today", "Config changes this week", "Referral decisions this month"

Dependencies: Phase A-2 (audit log data)

Success criteria:

1. All audit log entries are queryable and filterable
 2. Before/after state diffs render correctly
 3. Timeline view shows chronological activity
 4. Export produces valid CSV/JSON
 5. Performance: 10,000+ audit entries load and paginate smoothly
-

Workstream C: Experience-Based Recalibration

Scope Decisions (Confirmed)

- **Governance model:** Propose-and-approve — system proposes changes, human approves before deployment
- **This is the most architecturally significant workstream** — it closes the feedback loop between actual loss experience and model calibration

Phase C-1: Loss Data Ingestion & Signal-Loss Correlation

What: Schema and pipeline for ingesting actual loss events and linking them to DSI assessments, creating signal-to-loss pairs that the recalibration engine needs.

Backend scope:

- Loss event model: id, tenant_id, entity_id (assessed entity), policy_id, claim_reference, loss_date, notification_date, loss_type (frequency category), incurred_amount, paid_amount, reserved_amount, status (open/closed/reopened), coverage, config_name, cause_description, linked_assessment_id (FK to model version at bind time)
- Loss ingestion API: `/api/v1/losses/`
 - `POST /losses` — Create loss event (manual entry or bulk import)
 - `POST /losses/import` — Bulk CSV/JSON import
 - `PUT /losses/{id}` — Update loss (amount changes, status changes)
 - `GET /losses` — List with filtering by entity, coverage, date, status
- Signal-loss linker: When a loss event is created, automatically link it to the most recent assessment model version for that entity/coverage. This creates the signal-loss pair: the entity's signal profile at bind time paired with the actual loss outcome.

- Signal-loss pair table: `assessment_id`, `loss_event_id`, `signal_scores_at_bind` (JSONB — snapshot of all signal scores from the model version), `tier_at_bind`, `loss_propensity_at_bind`, `time_to_loss` (days between bind and loss event)
- Retrospective linker: Batch process to link historical loss events to historical assessments (for bootstrapping with existing data)

Frontend scope:

- Loss event entry form: Manual loss creation with required fields
- Bulk import interface: CSV upload with column mapping and validation preview
- Loss register: Table of all loss events with filtering, linked assessment details

Dependencies: None — can start immediately (backend only)

Success criteria:

1. Loss events can be created, updated, and queried
2. Each loss event is automatically linked to the assessment at bind time
3. Signal-loss pair table is populated with signal snapshots
4. Bulk import handles CSV with validation and error reporting
5. Retrospective linker correctly matches historical losses to historical assessments

Phase C-2: Recalibration Engine

What: The analytical engine that consumes signal-loss pairs and proposes config adjustments. This is the “propose” half of propose-and-approve.

Backend scope:

- Recalibration engine module: New module `infrastructure/recalibration/`
 - `engine.py` — Main recalibration orchestrator
 - `signal_analysis.py` — Per-signal predictive power analysis
 - `weight_optimiser.py` — Optimal weight computation
 - `tier_analysis.py` — Tier threshold analysis
 - `proposal.py` — Proposal generation and formatting

- `types.py` — Recalibration-specific types
- **Signal predictive power analysis:** For each signal in a coverage config, compute:
 - Discrimination: Does the signal score differ meaningfully between loss and no-loss entities? (Mann-Whitney U test)
 - Monotonicity: Does loss frequency/severity increase as signal score decreases? (Spearman correlation between signal score and loss outcome)
 - Stability: Is the signal's predictive power consistent across time periods? (Compute discrimination in rolling windows)
 - Contribution: Information value (IV) or Gini coefficient of the signal relative to loss outcome
 - Output: Per-signal report card with current weight vs evidence-supported weight
- **Weight optimisation:** Given the signal report cards, compute an optimal weight vector that maximises the composite score's discriminative power for loss prediction. Constrained optimisation: weights must sum to defined group totals, no weight can be negative, no single signal can exceed a maximum weight (prevents over-fitting to a single signal). Method: logistic regression coefficients normalised to the weight scale, or gradient-based optimisation with constraints.
- **Tier threshold analysis:** Given the loss data, evaluate whether current tier boundaries optimally separate loss frequency. If Tier 2 and Tier 3 have similar loss rates, the boundary should shift. Output: proposed threshold adjustments with statistical evidence.
- **Proposal generation:** Package all findings into a `RecalibrationProposal`:

```
class RecalibrationProposal:
    id: str
    coverage: str
    config_name: str
    proposed_at: datetime
    proposed_by: str # "system" for automated proposals
    status: str # draft, pending_review, approved, rejected, deployed

    signal_report_cards: list[SignalReportCard] # Per-signal analysis
    weight_changes: list[WeightChange] # Before/after for each signal
    tier_threshold_changes: list[TierThresholdChange] # Before/after

    impact_assessment: ImpactAssessment # What happens to current book
    statistical_evidence: dict # Full statistical provenance
```

```
reviewer: str | None
review_decision: str | None
review_rationale: str | None
reviewed_at: datetime | None
```

- **Impact assessment:** Before proposing, run the proposed weights against the current assessment database and compute:
 - Tier migration: How many entities would change tier?
 - Premium impact: What is the aggregate premium change?
 - Discrimination improvement: How much better does the new model separate loss from no-loss?
 - This ensures the reviewer can see the real-world impact before approving.
- **Scheduling:** The recalibration engine runs as a batch process, triggered when:
 - Loss data volume exceeds a threshold since last run (e.g., 50 new loss events)
 - Time since last run exceeds maximum interval (e.g., 90 days)
 - Manual trigger by actuarial user

Dependencies: Phase C-1 (loss data and signal-loss pairs)

Success criteria:

1. Signal report cards correctly identify high/low predictive signals in synthetic data
2. Weight optimiser produces valid weight vectors that respect constraints
3. Tier analysis correctly identifies suboptimal boundaries
4. Proposals include full statistical evidence and impact assessment
5. The engine runs end-to-end on seed data with synthetic losses

Phase C-3: Recalibration Governance UI

What: The “approve” half. A dedicated interface where the actuarial/product team reviews, approves, or rejects recalibration proposals before they are deployed.

Backend scope:

- Proposal API: `/api/v1/recalibration/`

- `GET /proposals` — List proposals with status filtering
- `GET /proposals/{id}` — Full proposal detail with evidence
- `POST /proposals/{id}/approve` — Approve with mandatory rationale
- `POST /proposals/{id}/reject` — Reject with mandatory rationale
- `POST /proposals/{id}/deploy` — Deploy approved proposal (updates config.yaml, runs calibration harness, creates new config version)
- `POST /proposals/{id}/simulate` — Run proposed weights against current book without deploying

Frontend scope:

- Recalibration dashboard: List of proposals with status badges (draft, pending, approved, rejected, deployed). Filterable by coverage, status, date.
- Proposal detail view: The core of the governance interface. Contains:
 1. **Signal Report Cards:** Per-signal analysis showing current weight, evidence-supported weight, discrimination score, monotonicity, stability. Traffic-light indicators (green = well-calibrated, amber = adjustment suggested, red = significant misalignment).
 2. **Weight Change Summary:** Table showing before/after weights for every signal that would change. Sortable by magnitude of change.
 3. **Tier Threshold Analysis:** Current vs proposed tier boundaries with loss rate overlay chart.
 4. **Impact Assessment:** Tier migration table (how many entities move between tiers), aggregate premium impact, discrimination improvement metric.
 5. **Simulation:** "What-if" button that runs the proposed weights against the current book and shows results without deploying.
 6. **Approval/Rejection:** Action buttons with mandatory rationale text field.
- Deployment flow: Approved proposal → Run calibration harness → Review results → Deploy (creates new config version via B-2 config management pipeline) → Audit trail records entire chain.
- Historical proposals: View past proposals (approved, rejected, deployed) with their outcomes and the actual loss performance after deployment.

Dependencies: Phase A-1 (auth — `recalibration:approve` permission), Phase A-2 (audit

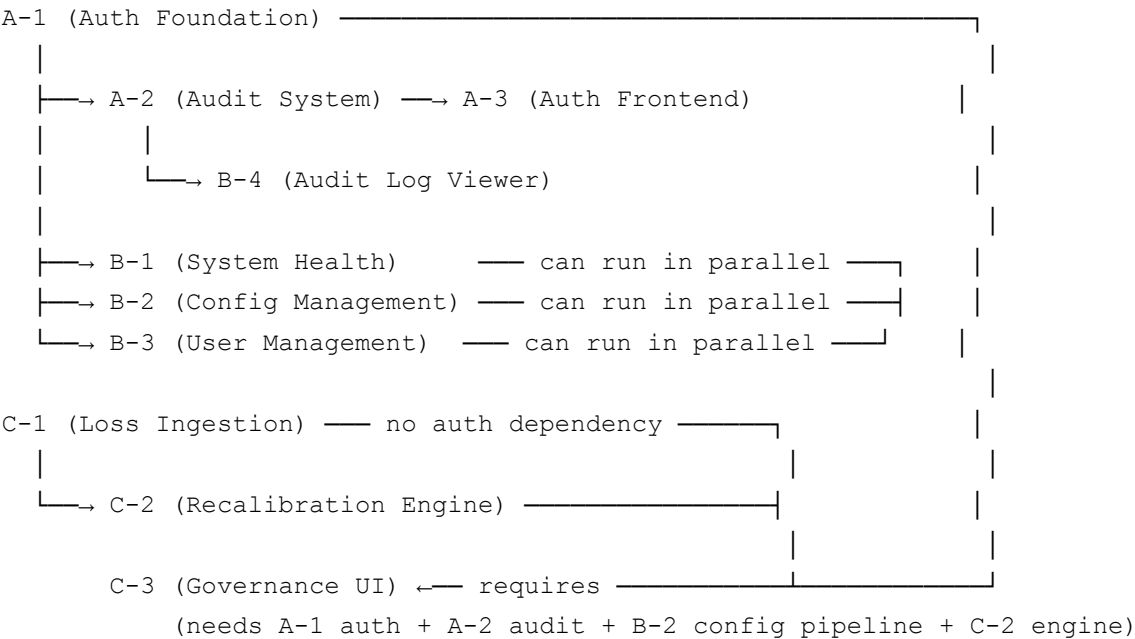
trail), Phase B-2 (config management pipeline for deployment), Phase C-2 (recalibration engine)

Success criteria:

- 1. Proposals display with full signal report cards and impact assessment
- 2. Simulation runs correctly and shows book-wide impact
- 3. Approval/rejection requires rationale and creates audit entry
- 4. Deployment updates config.yaml, runs calibration harness, and creates new config version
- 5. The complete chain from loss event → signal-loss pair → recalibration proposal → review → approval → deployment → new config version is traceable in the audit log
- 6. Non-actuarial users cannot approve proposals (permission-gated)

PART 4: EXECUTION PLAN

Dependency Chain



Recommended Build Sequence

--	--	--	--

Order	Phase	Rationale	Parallel?
1	A-1	Gates everything — auth must come first	No
2	C-1	Backend only, no auth dependency, enables C-2	Yes (with A-2)
3	A-2	Audit system needed for all subsequent phases	Yes (with C-1)
4	C-2	Backend only, needs C-1, enables C-3	Yes (with B-1, B-2)
5	B-1	System health dashboard — immediate operational value	Yes (with C-2, B-2)
6	B-2	Config management — needed for C-3 deployment flow	Yes (with B-1)
7	A-3	Auth frontend — can be deferred until backend phases stable	After A-1, A-2
8	B-3	User management — admin convenience, not blocking	After A-1
9	B-4	Audit viewer — consumes A-2 data	After A-2
10	C-3	Governance UI — the integration point, requires everything	Last

World Engine Integration

The 5 WE phases (already specified) fit alongside these workstreams:

- **WE-1** can start anytime (backend only, no auth dependency)
- **WE-2** can start after WE-1
- **WE-3** can start after WE-1 (parallel with WE-2)
- **WE-4** needs WE-3, should integrate CAF display into B-1 system health
- **WE-5** needs WE-3, portfolio dashboard should be permission-gated via A-1

Effort Estimates

Workstream	Phases	Backend	Frontend	Total

A: Auth & Audit	3	4-5 weeks	3-4 weeks	7-9 weeks
B: Admin Backend	4	5-6 weeks	6-8 weeks	11-14 weeks
C: Recalibration	3	4-5 weeks	3-4 weeks	7-9 weeks
WE: World Engine	5	10-13 weeks	3-4 weeks	14-18 weeks
Total	15			39-50 weeks

With parallelisation, the critical path is approximately 25-30 weeks.

APPENDIX A: Reference Phase Document Format

See the Phase WE-1 specification in the World Engine Phase Development Plan for the canonical example of a properly structured phase document. Key characteristics:

1. Every file to be created or modified is named with its full path
2. Code signatures include class names, method names, parameters, and docstrings
3. Database changes specify table names, columns, indexes, and migration file names
4. API endpoints specify method, path, request/response types
5. Frontend components specify file paths, data sources, content sections
6. Success criteria are specific and testable
7. Dependencies reference other phases by ID
8. Constraints state what must NOT change (e.g., "no changes to existing pricing logic")

APPENDIX B: DSI Principles That Must Be Followed

From the Foundational Principles document — these are non-negotiable across all phases:

1. **External Observability:** Primary signals must be obtainable without cooperation
2. **Machine Readability:** All signals must be analysable by automated systems
3. **Network Authority:** Relationship patterns reveal quality
4. **Behavioural Inference Over Self-Reporting:** Observable behaviour weighted over

stated intentions

5. **Absence as a Signal:** Missing expected signals is a measurable indicator
6. **Structured Data Utilisation:** Authoritative structured data incorporated directly
7. **Minimal Direct Inquiry:** Optional, constrained to 5-10 questions per model
8. **Organisational-Level Assessment:** Parent behaviour is a proxy for underlying assets
9. **Simplicity in Scoring:** Clear scoring flow, captured at atomic grain
10. **Agentic Readiness:** Models must be executable by autonomous systems

Additional critical principle: No subjective adjustments are permitted in the pricing formula. Underwriters audit and correct inputs (signals), not outputs (premiums). The only permitted subjective adjustment is the `offered_premium_discretion` which operates on the commercial output, not the technical model.

YAML is truth: Never hardcode values that exist in configuration.